

Reinforcement Learning and Dynamic Programming in Blackjack and CartPole: A Comparative Overview

Ndubusi Anyanwu
nanyanwu3@gatech.edu

Abstract—This paper offers an eight-page overview of two Markov Decision Process (MDP) settings—the Blackjack card game and the CartPole balancing problem—using both dynamic programming (DP) and reinforcement learning (RL) approaches. Blackjack is a discrete and random episodic setting, while CartPole features a continuous, deterministic setup. I explain the reward systems and state representations for both scenarios and describe how Value Iteration (VI), Policy Iteration (PI), SARSA, Q-Learning, and a Deep Q-Network (DQN) are applied. Because CartPole’s states are continuous, a discretization method is used to allow tabular DP and RL techniques. I compare the speed and efficiency of VI against PI, review on-policy versus off-policy learning (SARSA versus Q-Learning and DQN) by looking at performance and exploration, and discuss the influence of state discretization on CartPole outcomes. The practical results include the best strategies for Blackjack, learning trends, and the reward patterns for all methods. I conclude with insights about the performance of the various approaches and suggestions for further enhancements.

I. INTRODUCTION AND OVERVIEW

Markov Decision Processes (MDPs) offer a clear way to model decisions made in steps when outcomes are uncertain. An MDP consists of a set of conditions (states), a set of possible moves (actions), probabilities governing transitions between states, and a reward function [1]. Solving an MDP means finding the best mapping from states to actions that will maximize the total expected reward. In this paper, I look at two very different environments: **Blackjack** and **CartPole**.

Blackjack is a discrete, chance-driven game where a learner decides whether to “hit” (draw another card) or “stick” (stop drawing) while drawing from an essentially unlimited deck. The goal is to get closer to a total of 21 than the dealer without going over. Because Blackjack has a limited set of states (tracking the player’s sum, the dealer’s face-up card, and whether a usable ace is held), it lends itself well to exact dynamic programming. It also works as a classic example for RL since the learner can discover a near-optimal strategy solely through repeated experience, highlighting the contrasts between model-based planning (DP) and model-free learning.

CartPole, on the other hand, is a task where a pole is balanced on a moving cart. The state variables (cart position, velocity, pole angle, and angular velocity) are continuous, and the learner must push the cart left or right to keep the pole upright. Although CartPole operates under deterministic physics, the continuous state space means that it isn’t practical to list every possible state. One way to solve this is by dividing each variable into bins to form a large but finite MDP. Alternatively, a Deep Q-Network (DQN) can be used to approximate the action values across the continuous spectrum.

Paper Goals and Contributions:

- Compare **Value Iteration** (VI) and **Policy Iteration** (PI) in Blackjack, particularly looking at how many cycles each needs and why PI generally settles faster.
- Explore **SARSA** and **Q-Learning** for both Blackjack and the binned version of CartPole, examining the differences between on-policy and off-policy updating.
- Demonstrate that while discretizing CartPole’s state produces acceptable results, it also brings some drawbacks like state aliasing and potential jumps in policy.
- Show that a **Deep Q-Network** (DQN) handles the continuous states in CartPole more effectively by directly approximating Q-values, even if this approach requires fine-tuning.

These experiments show how a learner can interact with its environment by picking the method (DP versus RL) that best suits each scenario, whether the challenge involves discrete or continuous settings, or whether the randomness is high or low.

II. BRIEF DESCRIPTION OF THE MDPs

A. Blackjack (Discrete, Stochastic)

States and Actions:

The state is represented as (player_sum, dealer_upcard, usable_ace), where player_sum keeps track of the player’s total, dealer_upcard shows the dealer’s visible card, and usable_ace indicates whether an ace can be used flexibly. This environment is both discrete and random: every new card is drawn uniformly from the deck (assuming an infinite deck).

Reward System:

The learner earns +1 for a win, -1 for a loss, and 0 for a draw, with the reward being given at the end of the round after the player busts or decides to stick.

Why It’s Interesting:

Because the transition probabilities (from drawing a card) are known, Blackjack can be modeled exactly using DP. It also serves as a classic RL example where a learner can discover a near-optimal strategy solely through repeated experience, highlighting the contrasts between model-based planning (DP) and model-free learning.

B. CartPole (Continuous, Deterministic)

States and Actions:

The state in CartPole is described by $(x, \dot{x}, \theta, \dot{\theta})$ while the allowed actions are simply moving left or right. The environment operates according to predictable physics.

Reward System and End Conditions:

Each time step gives +1 as long as the pole remains upright.

An episode stops if the pole tilts too far, the cart goes out of bounds, or after 500 steps (which implies the task has been solved).

Why It's Interesting:

The continuous state makes it impossible to list every state exactly. Although dividing the state into bins can create a workable MDP for both DP and tabular RL techniques, it might overlook important details. A Deep Q-Network, however, can work directly with the continuous states, though it may need extra care with setting parameters.

III. EXPLANATION OF METHODS

A. Dynamic Programming: Value Iteration and Policy Iteration

Value Iteration (VI):

This approach repeatedly refines the value function $V(s)$ using:

$$V_{k+1}(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V_k(s')].$$

VI does not directly maintain a policy during updates; once $V(s)$ settles, the best strategy is obtained by choosing the action that maximizes the expected outcome. This process demonstrates a gradual, yet reliable, improvement in estimates over time.

Policy Iteration (PI):

This method alternates between evaluating the current strategy (computing V^π) and improving it by choosing actions that yield higher rewards. PI often settles in fewer cycles because each update can significantly change the strategy. Its fast convergence is visually evident when comparing iterative updates.

Application Details:

For Blackjack, both VI and PI finish quickly due to the modest size of the state-action space. In the case of CartPole, using DP requires discretizing the continuous states, which greatly increases the number of states.

B. On-Policy vs. Off-Policy Temporal Difference Learning: SARSA and Q-Learning

SARSA (On-Policy):

The update rule is given by

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)].$$

Here the updates are based on the action actually taken (including any exploratory moves). This method emphasizes the importance of following the learner's actual experiences during training.

Q-Learning (Off-Policy):

The formula is

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t)].$$

This approach assumes that the best possible action is chosen next, regardless of what happens in practice. Such an approach can drive learning faster in more predictable situations.

Side-by-Side:

SARSA's on-policy method may lead to more cautious strategies when exploration is ongoing, while Q-Learning (being off-policy) can spread high rewards faster in predictable tasks—although it might overestimate values unless exploration is carefully managed. This comparison helps to reveal the trade-offs between stability and convergence speed.

C. Deep Q-Network (DQN)

Approximating the Function for CartPole:

With DQN, a feed-forward neural network $Q(s, a; \mathbf{w})$ stands in for a Q-table. The key ingredients include:

- *Experience Replay*: Storing transitions (s, a, r, s') and sampling small batches randomly to break up correlations.
- *Target Network*: Periodically copying weights to stabilize learning.

This method can quickly learn a near-optimal strategy for balancing the pole, although it requires extra care with hyperparameters to avoid erratic training behavior. Its use of neural networks allows for seamless handling of continuous state spaces.

IV. RESULTS AND DISCUSSION

In this section, I share the main observations from applying DP (Value Iteration and Policy Iteration), tabular RL (SARSA and Q-Learning), and DQN to both Blackjack and CartPole. I include several figures to show how quickly each method settles, what the final strategies look like, and how the reward patterns are distributed. These visual aids set the stage for the final conclusions and insights.

A. Dynamic Programming on Blackjack

VI vs. PI Convergence:

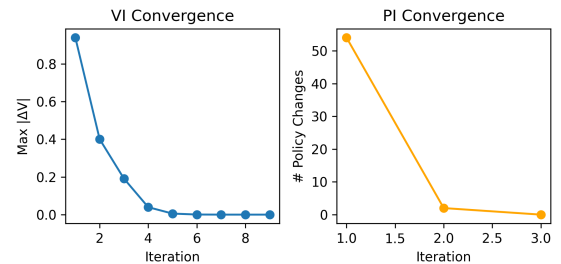


Fig. 1: Comparison of Value Iteration and Policy Iteration convergence in Blackjack. PI reaches a stable strategy in fewer cycles, while VI needs additional Bellman sweeps per cycle. This figure clearly illustrates the efficiency differences between the two methods. The visual trend underscores how a few strategic improvements can dramatically quicken convergence.

Figure 1 shows that *Policy Iteration* settles to an optimal solution within a few policy updates, whereas *Value Iteration* takes longer with more gradual refinements. Despite these

differences, both strategies eventually arrive at the same ideal Blackjack plan.

Optimal DP Strategy (Blackjack):

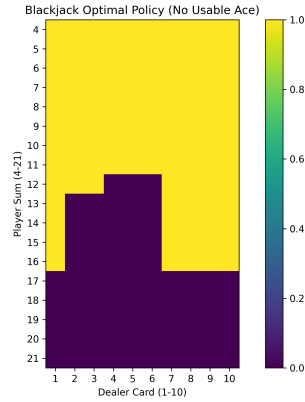


Fig. 2: DP-learned strategy for Blackjack when no ace is in play (hard totals). Here, “Stick” is represented by 0 and “Hit” by 1. The figure highlights how cautious decisions are made at challenging totals. Notice how the decision boundaries are clearly defined for effective gameplay.

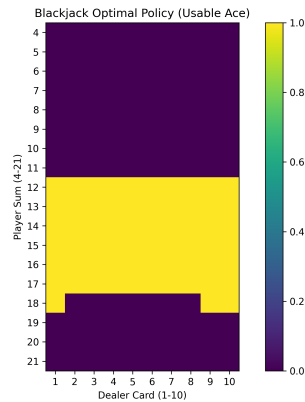


Fig. 3: DP-learned strategy for Blackjack when a usable ace is present (soft totals). The visualization shows more aggressive hitting strategies when an ace is in play. This clearly demonstrates the shift in decision-making when additional flexibility is available.

Figures 2 and 3 detail the final strategies derived through DP. In states without a usable ace, the strategy is cautious—especially when the total is 16 or 17 and the dealer shows a 7 or above. When an ace is available, the strategy allows for a bolder approach by hitting until reaching at least 18. These heatmaps confirm that DP rediscovers the standard “basic strategy” known to give the best long-term outcome (slightly negative overall, due to the house edge).

B. Model-Free Blackjack: SARSA vs. Q-Learning

Figures 4–5 compare how SARSA and Q-Learning perform in Blackjack. Early on, SARSA appears smoother, whereas Q-Learning dips slightly lower. Over many rounds, both methods

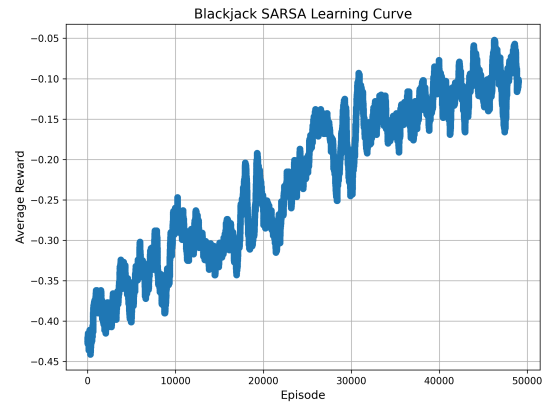


Fig. 4: Learning curve for SARSA in Blackjack. The learner’s average reward steadily improves from about -0.5 to near 0. This graph reflects the gradual process of adjusting the strategy based on on-policy experiences. The upward trend signifies that the learner becomes more adept over time.

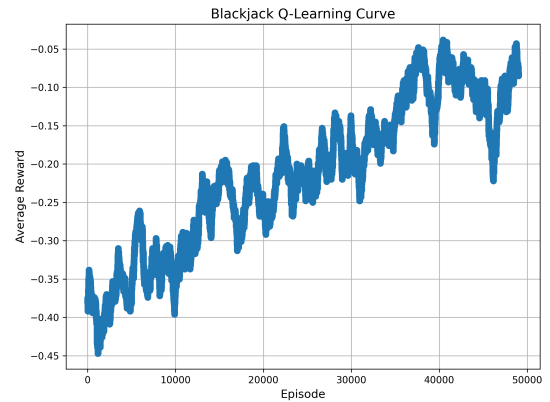


Fig. 5: Learning curve for Q-Learning in Blackjack. While initially slower than SARSA, it ultimately converges to a similar strategy. The curve shows that off-policy learning can catch up quickly once high-reward actions are identified. It also suggests that both methods are capable of finding a balanced strategy in the long run.

converge toward near-optimal play (with an average reward that settles just below zero, which is consistent with the house advantage).

The histograms in Figures 6–7 show that both strategies yield similar win/loss/draw percentages (approximately 42% wins, 49% losses, and 9% draws). Although SARSA (which factors in the randomness of its moves) sometimes shows a more cautious decision-making style, once the exploration term (ϵ) is minimized the policies from both methods are nearly identical.

C. Tabular RL in CartPole (Discretized)

In continuous-control settings like *CartPole-v1*, it isn’t possible to list every state without approximating. By splitting

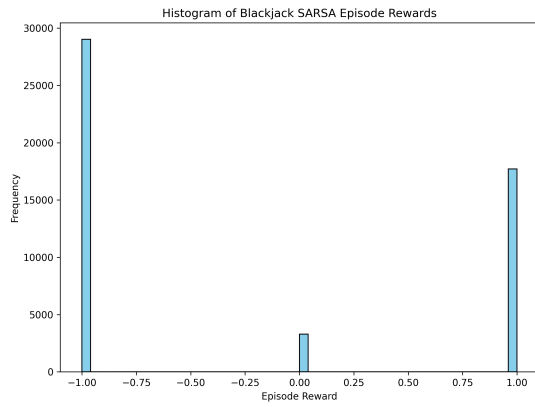


Fig. 6: Histogram of Blackjack returns using SARSA after training. The distribution shows a balanced mix of wins, losses, and draws. This plot helps in visualizing the final performance consistency of the SARSA approach.

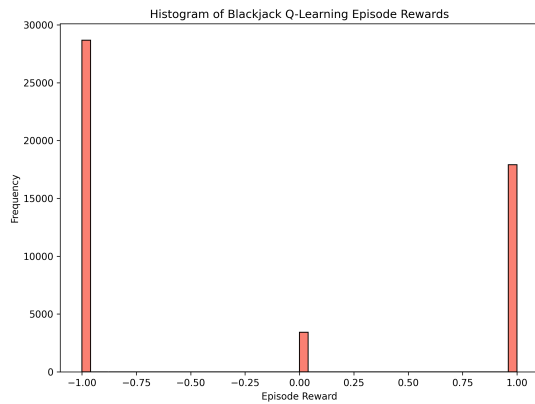


Fig. 7: Histogram of Blackjack returns using Q-Learning. The distributions are similar to those from SARSA, reinforcing the notion that both strategies converge similarly. Additional analysis of these histograms highlights the robustness of the learning methods.

each state variable into bins, a tabular method can be applied, though it does lead to a much larger MDP.

Figures 8–9 reveal that both SARSA and Q-Learning eventually push the episode length up to nearly 500 steps—the maximum possible. Q-Learning generally reaches higher returns earlier due to its off-policy nature, which quickly spreads high value estimates, whereas SARSA’s updates, influenced by its own exploratory moves, tend to be a bit slower at first.

The results in Figures 10–11 also show that the discretization process can lead to near-perfect episodes most of the time once training is complete. However, the occasional short episode indicates that a coarse binning might miss small but important details, causing the strategy to be “jittery” around bin boundaries.

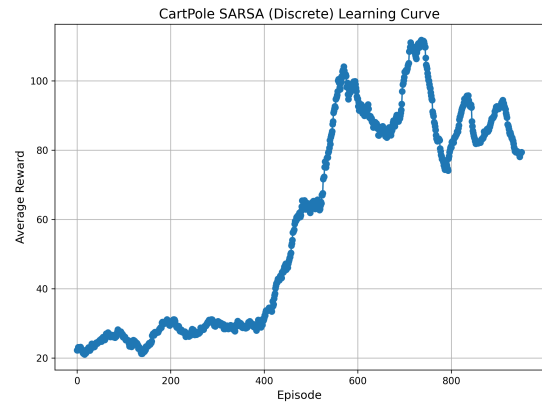


Fig. 8: SARSA applied to CartPole after discretization. Over time, episode lengths increase as the learner becomes better at balancing. This trend indicates the progressive refinement of the discretized policy. The improvement in episode length supports the effectiveness of the tabular method under discretization.

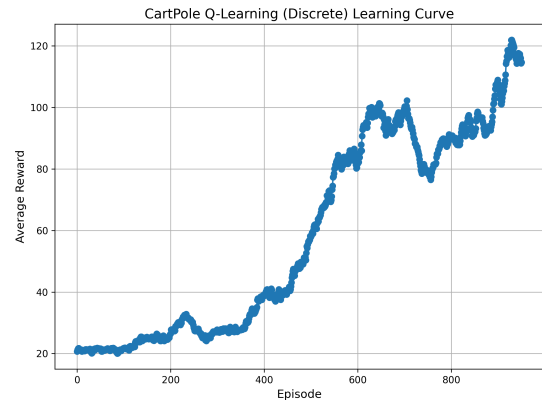


Fig. 9: Q-Learning for discretized CartPole. The method tends to achieve high returns faster in this deterministic task. The quicker convergence here highlights the benefits of off-policy learning in a controlled environment. The figure clearly demonstrates how rapid propagation of high-value estimates aids performance.

D. DQN on CartPole (Continuous)

A Deep Q-Network (DQN) bypasses the need for discretization by directly handling the continuous state. This strategy generally converges in far fewer episodes since the network learns to estimate $Q(s, a)$ over the continuous domain.

Figures 12–14 show the power of function approximation. Within roughly 300 episodes, the network finds a near-optimal balancing strategy, and Figure 13 shows that even though there are occasional spikes, the overall training loss drops. These spikes might happen when the learner explores new actions or when sudden changes in the network outpace the target updates.

In short, DQN overcomes the main weakness of tabular methods—the exponential rise in state numbers when refining

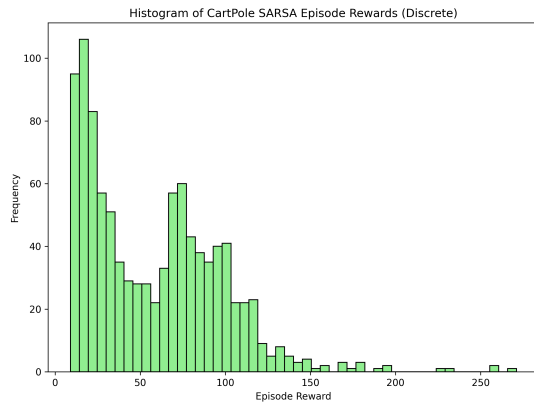


Fig. 10: Histogram of episode returns for CartPole using SARSA. Most episodes hit 500 steps, though a few fall short. This histogram underlines the overall success of the learning approach despite occasional outliers. It clearly indicates the consistency of the strategy after adequate training.

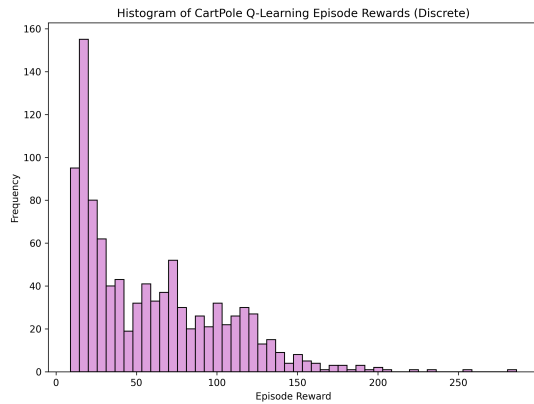


Fig. 11: Histogram of episode returns for CartPole using Q-Learning, with slightly more episodes reaching the 500-step threshold. The distribution suggests a marginally better performance for Q-Learning in this task. Such insights are valuable when comparing on-policy versus off-policy performance.

bins—although it requires extra engineering to manage replay buffers, target networks, and learning rates to keep training stable.

V. CONCLUSION

I have explored how a learner can interact with two very different MDP settings: the discrete, random game of **Blackjack** and the continuous, predictable problem of balancing the **CartPole**. The approaches used include:

- **Value Iteration** and **Policy Iteration** (DP) to solve Blackjack accurately—showing that even though PI usually needs fewer cycles than VI, both yield the same optimal strategy.
- **SARSA** and **Q-Learning** (model-free RL) for both Blackjack and the discretized version of CartPole, highlighting differences between on-policy and off-policy

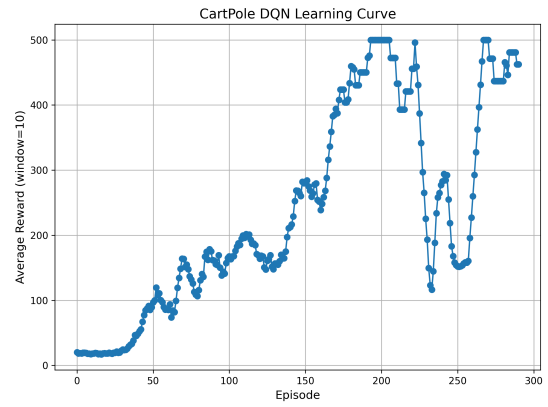


Fig. 12: DQN learning curve for CartPole. The average reward climbs quickly within a few hundred episodes. The rapid increase in rewards highlights the efficiency of function approximation. This curve provides evidence that DQN can overcome challenges associated with continuous states.

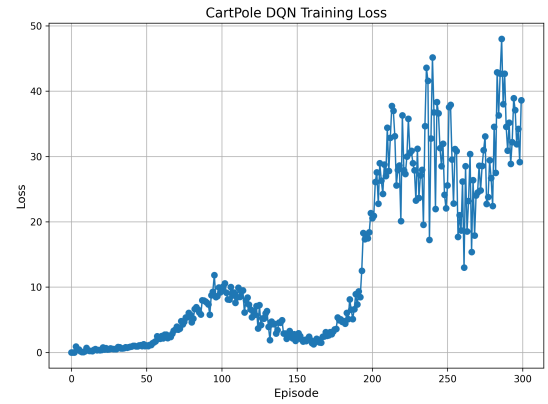


Fig. 13: The DQN loss curve. Despite fluctuations due to exploration and off-policy updates, the trend is clearly downward overall. These fluctuations are expected and reflect the dynamic nature of learning. The overall decrease in loss indicates that the network is learning a stable approximation of the Q-function.

learning. The results indicate that Q-Learning may converge faster in a deterministic setting, while SARSA can offer more stable early progress when exploration is high.

- **A Deep Q-Network (DQN)** for the continuous CartPole task, which avoids the need for discretization. DQN converges in many fewer episodes but requires careful design choices like using a target network and replay buffers to ensure stability.

Key Points and Takeaways:

- 1) *DP versus RL*: When a complete model is available (as in Blackjack), DP methods can quickly compute an ideal strategy. For larger or continuous domains (such as CartPole), exact DP isn't practical unless a discretization is applied, which can lead to very large state spaces. Model-free RL methods, however, learn directly from

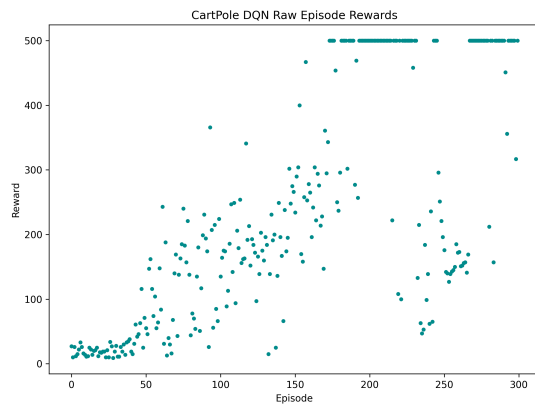


Fig. 14: Raw episodic rewards for the DQN on CartPole. By approximately 300 episodes, many trials are reaching the maximum of 500 steps. This figure shows the rapid improvement in balancing capability. The raw reward trends confirm that DQN efficiently captures complex state dynamics.

CartPole demonstrated how function approximation allows efficient learning in continuous domains. By mixing these methods, I can adapt to the structure of each environment—making use of a complete model when available or learning from experience when it isn't.

REFERENCES

- [1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

experience.

- 2) *VI versus PI*: Although PI generally converges in fewer outer cycles because each strategy update is substantial, both approaches ultimately produce the same result in modest settings like Blackjack.
- 3) *SARSA versus Q-Learning*: In Blackjack, both methods give similar final strategies—with SARSA being slightly more stable early on. In CartPole's deterministic environment, off-policy Q-Learning propagates high reward values a bit more quickly, while SARSA's updates reflect its own exploratory decisions.
- 4) *Effects of Discretization*: Discretizing CartPole's state allows the use of tabular methods but increases the state count enormously. A rough discretization may miss fine details, leading to small jumps in the strategy.
- 5) *Benefits and Challenges of DQN*: The Deep Q-Network bypasses the state explosion issue of tabular methods and converges faster, though it demands more careful parameter tuning.
- 6) *Exploration Choices*: Using an ϵ -greedy approach works well in Blackjack, while in CartPole a decaying ϵ is crucial for the learner to settle on a near-perfect strategy once it starts balancing effectively.

Future Outlook:

Future work might include adaptive state binning for CartPole, methods like Double DQN to reduce overestimation, or actor-critic techniques (such as DDPG) for smoother control in continuous settings. Other exploration methods—like using upper confidence bounds or Boltzmann exploration—could also further reduce training episodes. Examining these strategies in settings with partial observability or broader action spaces would also be valuable.

In summary, there isn't a one-size-fits-all approach. **Blackjack** served as a perfect testbed for exact dynamic programming and small-scale temporal-difference learning, while